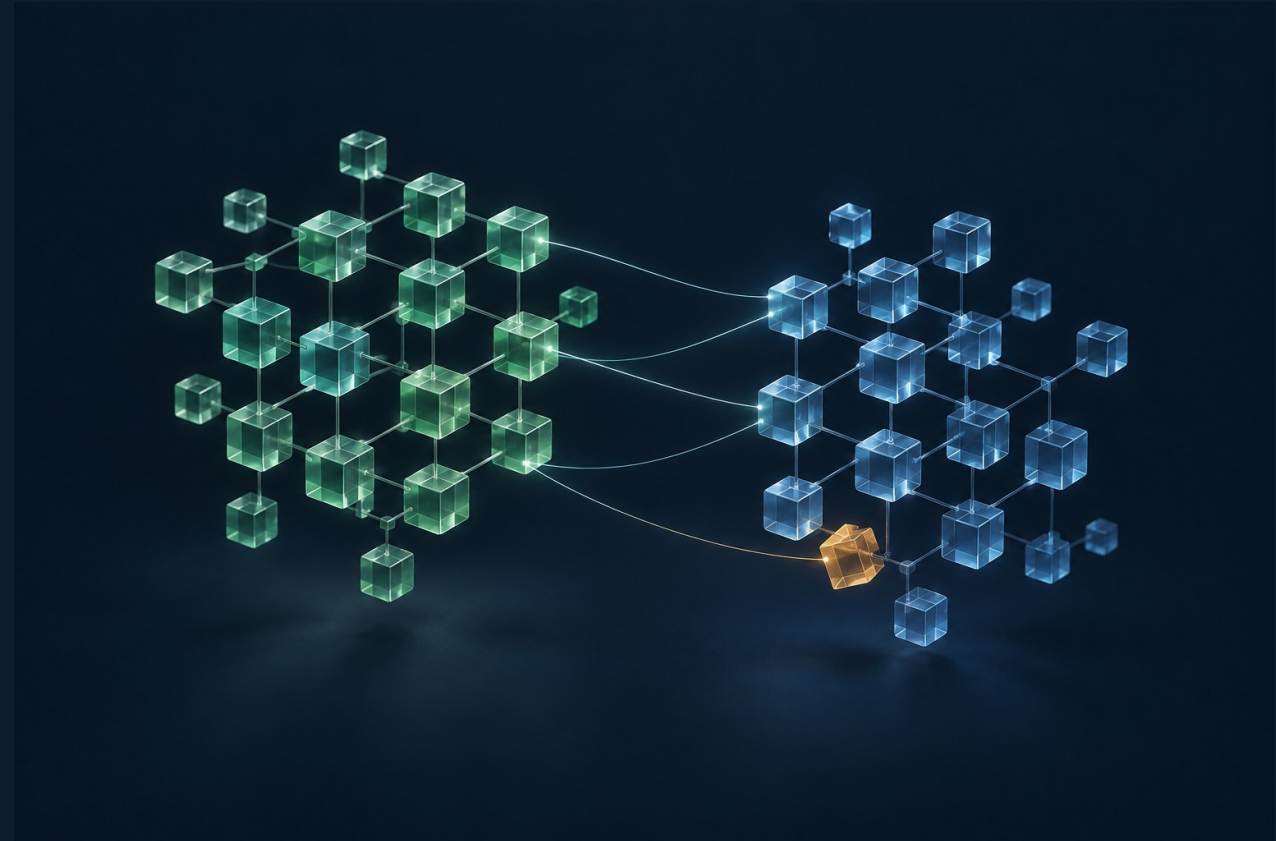


SAGE

Scientific code translation with auditable evidence

Rust and C++20 translations
Completed checks and next steps

Updated October 6, 2026



Why scientific translation needs evidence

Code can compile and still change the science.

What can change	Example consequence
Layout and ownership	A transpose, stride or alias changes the result.
Numerical behavior	Rounding, conditioning or basis choices alter comparisons.
API and failure behavior	Shapes, exceptions or lifetimes break downstream use.

Review needs a replayable result tied to source, inputs and a stated policy.

What SAGE does

- | | |
|-------------------------------|---|
| 01 Capture the target | Pin source, domain, policy and build environment. |
| 02 Produce a candidate | Snapshot a checked-in Rust or C++20 candidate. |
| 03 Generate cases | Accept new, valid, unique SymSan outputs. |
| 04 Compare behavior | Execute source, candidate and an independent NumPy reference. |
| 05 Preserve evidence | Record outcomes and replay discrepancies in a run bundle. |

How SAGE relates to Einsums

Einsums is the scientific library under evaluation.

Component	Role in this project
Einsums C++ / Python	Pinned source implementation and compatibility reference.
Independent Rust port	Broad CPU/Python surface via einsums / pyeinsums.
Independent C++20 port	Broad CPU/Python surface backed by independent C++20.
SAGE / SymSan / NumPy	Generate cases, compare candidates and retain evidence.

Pinned Einsums v1.1.5. Both ports cover CPU/Python. HIP/GPU remains excluded.

Rust and C++20 CPU/Python translations

90

CPU Python exports

16

public utility helpers

4

real / complex dtypes

Tensors and contractions
Linear algebra and FFT

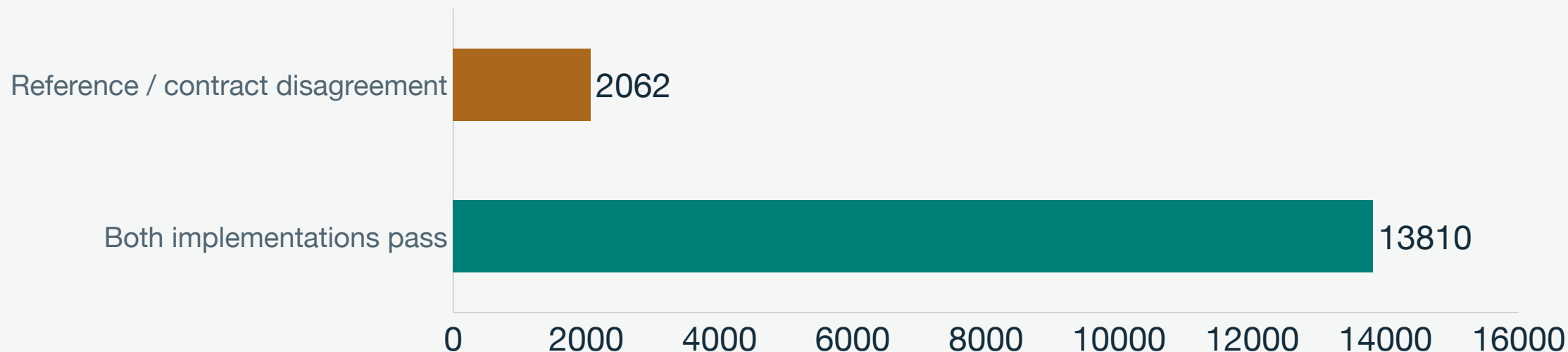
Decomposition and storage
HDF5 and runtime services

Each library profile snapshots a checked-in independent port.

Export coverage records implementation. Exact compatibility remains incomplete.

Saved Rust replay results

128 inputs × 31 API groups × 4 dtypes = 15,872 checks



The Rust candidate met every replay numerical check.

Corpus replay expands API checks without adding new symbolic paths.

Rust replay saved September 8, 2026. The full-library gate remains INCOMPLETE_SCOPE.

How the comparison is interpreted

Source / candidate / reference	Interpretation
All agree within policy	Observed agreement for that case and contract.
Candidate differs, source agrees	Candidate discrepancy to replay and diagnose.
Source fails, candidate meets policy	REFERENCE_DISAGREEMENT; review source contract and oracle.
Shape, error or execution differs	Preserve structural differences, exceptions and timeouts.

A mathematical correction may intentionally break source compatibility.

C++20 translation and checks

Independent port

90 core exports, 16 helpers and the broad CPU/Python modules.

Fresh SymSan cases

128/128 PASS against Einsums and NumPy across 23 operations, through native and Python calls.

API replay and native grid

15,872 replay checks, 2,056 reference disagreements. The native grid passes 4,416 cases.

Ownership and safety

Views, runtime and storage tests pass. Native contracts pass ASan/UBSan.

129 project tests pass. Installed C++ headers and Python wheel checks pass.

SymSan campaign boundary

Execution layer	Current treatment
Input decoder	Instrumented by SymSan; operation and dimension branches.
Einsums C++ computation	Runs natively; internal library paths are not explored.
Rust / C++20 candidates	Concrete replay and comparison with NumPy.
New API / dtype combinations	Broader replay checks, with no new concolic paths.

Saved Rust: 128 inputs. Fresh C++20: 128 accepted inputs.

Both use decoder instrumentation. Internal-library branch coverage remains pending.

The contract determines what “same” means

Contract layer	Comparison policy
Structure and behavior	Shapes, dtype, strides, mutation, errors and termination.
Finite numerical values	Absolute / relative tolerance with explicit dtype rules.
Nonunique numerical results	Reconstruction, residuals and orthogonality.
Machine semantics	Rounding, FMA, NaNs, signed zero and defined memory access.

Compatibility and mathematical correctness are separate claims.

The main gaps

Gap	Why it limits the claim
SymSan reach and diversity	Campaigns omit internal paths and reuse one numeric payload.
Compatibility contracts	Rust: 2,062 and C++: 2,056 replay disagreements need review.
Validation breadth	General shapes/dtypes, upstream BLAS and full adapter proofs remain open.
Calling contracts and automation	Exact C++ overload/ABI compatibility and automatic library translation remain open.
Adoption and performance	Psi4 compatibility and performance equivalence remain unestablished.

A bounded verification model for translation

Source and candidate satisfy the same exact specification.

$$\text{Bad}(x) = P(x) \wedge \text{DefinedSource}(x) \wedge [\text{UnsafeCandidate}(x) \vee \neg R(\text{Source}(x), \text{Candidate}(x))]$$

P = allowed inputs R = the declared relation between observable results

SAT Replay a concrete counterexample.

UNSAT No counterexample within the complete bounded model.

Unknown / timeout No proof. Retain assumptions and solver outcome.

Rust and C++20 bounded proofs

Copy, add, multiply, transpose, matmul and scale

Rust proof	144 Kani proofs of ordered expressions from production methods.
C++20 proof	144 CBMC expression proofs with compiler identity checks.
Public API connection	Eligible calls use proved methods. 864 Python and 864 JSON/FFI checks per language.
Fault controls	Six source faults per language detected and replayed as native failures.

Float64, dimensions 1–4, signed bytes / 8. Adapters and upstream BLAS excluded.

Next steps and the evidence each should produce

Work item	Reviewable result
1 Shared contracts	Classify 2,062 Rust and 2,056 C++ replay disagreements.
2 Calling contracts	Review full Python calls, native C++ overload/ABI and installation contracts.
3 Broader verification	Prove public adapters, extend shapes/dtypes, then verify upstream BLAS contracts.
4 Application validation	Downstream regressions and native performance benchmarks.

Both ports still need evidence for the complete library contract.

Appendix: Rust reference disagreements

API group	Checks	Interpretation to investigate
Pseudoinverse	512	Shape / threshold and mathematical contract.
Truncated SVD	512	Deterministic k modes versus upstream projected modes.
Truncated symmetric eigen	484	Truncation contract and randomized behavior.
Continuous Lyapunov	256	Complex backend support differs.
LU / nullspace / general eigen	298	232 / 43 / 23; factor, threshold or residual differences.

Rust replay: 2,062 disagreements. Separate C++ replay: 2,056, documented in notes.

Appendix: where verification tools fit

Tool / approach	Role	Main boundary
SymSan	Generate inputs from path constraints.	Decoder only. Source computations run natively.
Alive2	Check LLVM function-pair refinement.	Requires compatible IR / memory model; bounded loops.
CBMC + Kani	Six bounded compositional Tensor proofs.	Trusted math/compiler/memory. Adapters/BLAS excluded.
SAGE	Translate, replay and retain both kinds of evidence.	Whole-library equivalence remains incomplete.

Tool choice follows the kernel, semantics and proof obligation.